

Instituto Superior de Engenharia de Lisboa
Licenciatura em Engenharia Informática e de Computadores
Programação Concorrente
Exame de Época de Recurso, Verão de 2024/2025
Duração: 2 horas

1. [2] Considere a classe **UnsafeModuloCounter** com uma implementação não *thread-safe* de um contador módulo. Realize uma versão *thread-safe* desta classe, com recurso ao uso de *locks*.

```
class UnsafeModuloCounter(  
    private val modulo: Int,  
) {  
    init { require(modulo > 1) }  
    private var counter: Int = 0  
    fun increment(): Int {  
        counter += 1  
        if (counter == modulo) {  
            counter = 0  
        }  
        return counter  
    }  
}
```

2. [6] Realize a classe **FixedThreadPool**, que implementa um *thread pool* com um número fixo de *threads*.

```
class FixedThreadPool(  
    nOfThreads: Int,  
) {  
    init { require(nOfThreads >= 1) }  
    @Throws(RejectedExecutionException::class)  
    fun execute(runnable: Runnable): Unit  
    fun shutdown(): Unit  
    @Throws(InterruptedException::class)  
    fun awaitTermination(): Unit  
}
```

As *worker threads* devem ser criadas quando a instância de **FixedThreadPool** é criada e só devem terminar após ser realizado o primeiro *shutdown*. A chamada ao método **shutdown** coloca o executor em modo de encerramento e retorna de imediato. Neste modo, todas as chamadas ao método **execute** deverão lançar a exceção **RejectedExecutionException**. Contudo, todas as submissões para execução feitas antes da chamada ao método **shutdown** devem ser processadas normalmente.

O método **awaitTermination** permite a qualquer *thread* invocante sincronizar-se com a conclusão do processo de encerramento do *thread pool*, isto é, aguarda até que sejam executados todos os comandos aceites e que todas as *worker threads* activas terminem. Este método deve suportar o protocolo de interrupções da plataforma Java.

3. [4] Realize o sincronizador **UnaryCapacityStream**, para utilizar em corrotinas, que representa uma sequência de elementos do tipo **T**.

```
class UnaryCapacityStream<T> {  
    suspend fun add(item: T): Int  
    suspend fun read(index: Int): Pair<Int, T>  
}
```

A função **add** adiciona um elemento à sequência, retornando o índice do elemento adicionado. Internamente, o sincronizador apenas tem capacidade para um elemento, pelo que esta adição pode eliminar o elemento de índice anterior, caso exista. A função **read** recebe o índice do elemento a ler e retorna um par com: o índice do elemento retornado, maior ou igual que o índice pretendido; e o elemento com o índice retornado. A execução da função **read** deve esperar até que o índice pretendido ou um índice superior a este esteja disponível.

4. [5] Realize a seguinte função *non-suspend*

```
fun <T1, T2> both(f1: suspend ()->T1, f2: suspend ()->T2, timeout: Duration)  
    : Pair<T1, T2>?
```

A função **both** avalia as funções **f1** e **f2**, em paralelo e tirando partido de ambas serem funções *suspend*. Retorna um par com o resultado de ambas as avaliações; ou **null**, caso o tempo máximo para a avaliação tenha decorrido. Este tempo é definido pelo parâmetro **timeout**. Esta função deve usar concorrência estruturada, garantindo que quando retorna não existem corrotinas por ela criada ainda em execução.

5. [3] Realize a função

```
suspend fun <T,R> Executor.map(f: (T)->R, input: List<T>): List<R>
```

Esta função promove a avaliação da função **f** sobre todos os elementos de **input**, no *thread pool* representado pelo **Executor** passado como receptor, tirando partido das múltiplas *worker threads* que este pode ter. O resultado é uma lista com o resultado dessas avaliações. Os elementos da lista resultado devem refletir a ordem dos elementos na lista de entrada, i.e., o elemento no índice **i** da lista retornada deve ser o resultado da avaliação da função **f** sobre o elemento de índice **i** na lista de entrada, para qualquer índice **i** válido. A função **map** não deve bloquear a *thread* onde é chamada. Assuma que a avaliação de **f** nunca produz exceção. A solução não deve usar funcionalidades presentes no *package* **kotlinx.coroutines**.

Duração: 2 horas
ISEL, 3 de julho de 2025